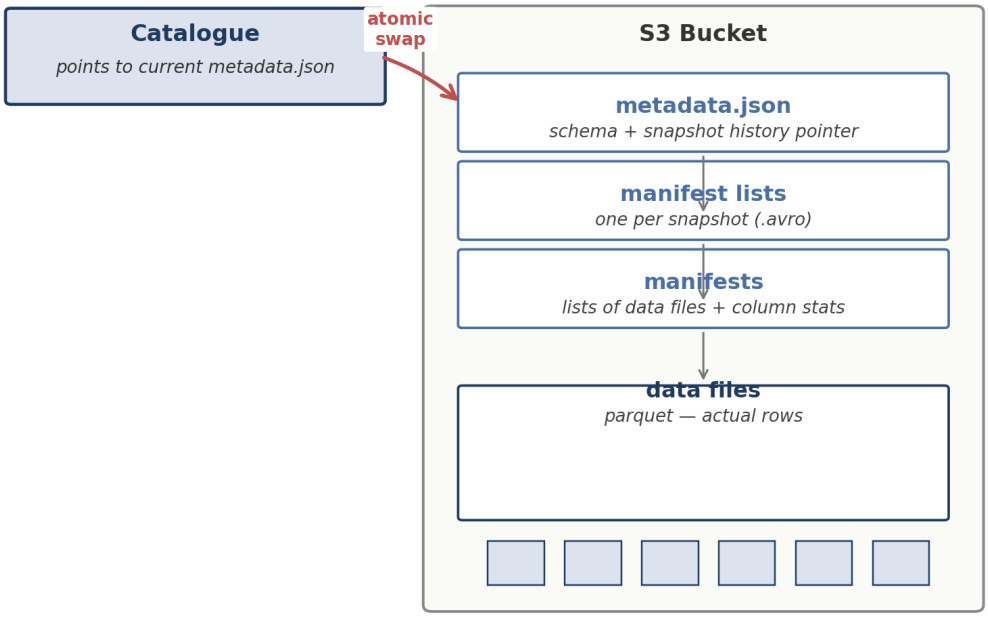
Data Platform Engineering

# Iceberg on S3

*An operator's guide to owning the table format — and what that means for the Snowflake line item*




Prepared by the Data Platform team

*April 2026 · Draft for management review*


## ABSTRACT

Apache Iceberg is an open table format that lets us keep our data in S3 and query it with any compatible engine — including Snowflake. Moving off Snowflake's fully-managed warehouse onto self-managed Iceberg promises meaningful cost savings, but those savings are conditional: whoever owns the tables inherits a continuous maintenance workload that Snowflake quietly handles today. This paper walks through the operational mechanics of Iceberg, surveys the five realistic catalogue options, presents an illustrative five-year cost model, and ends with a migration-readiness checklist. The headline finding: the cost case for leaving Snowflake is real, but only once the engineering team has committed to — and budgeted for — the maintenance jobs summarised in §4.

# 1. Executive summary

**Iceberg is a file layout, not a database.** It is a specification for organising Parquet files in S3 so that a set of them behaves like a table with transactions, schema, and history. The format itself is free; what costs money is the *maintenance* that keeps an Iceberg table healthy.

**Snowflake's value is not its query engine.** It is the silent, continuous maintenance it performs on every table: compaction, clustering, statistics, snapshot retention, garbage collection, and concurrency coordination. Teams that move off Snowflake inherit each of these jobs or accept a gradual performance degradation.

**The cost comparison is not "Snowflake vs. S3."** It is Snowflake versus (S3 + compute + headcount + incident exposure + tooling). Our illustrative model (Figure 4) shows a break-even around year two for a 50 TB workload, with material savings thereafter — *conditional on the team actually operating the platform at Snowflake's service level.*

**There is a middle path.** AWS S3 Tables and Databricks Unity both run the maintenance jobs for you while keeping the data in the open Iceberg format. They capture most of the portability benefit without the operational burden of fully self-managed Glue or Polaris/Nessie.

### Recommendation

Pursue a dual-path pilot — AWS S3 Tables as the managed control, a self-managed Glue path as the cost-optimised variant — and decide after measuring compaction behaviour, query latency, and real engineering time on both for one quarter. The repository already has both paths wired up (§6).

| Key terms | | |
|---|---|---|
| **Catalogue** — the service that knows which *metadata.json* is current for a given table. | **Snapshot** — an immutable version of the table. Every write produces one. | **Compaction** — rewriting many small files into fewer, larger files so reads stay fast. |
| **Optimistic concurrency** — two writers may run in parallel, but only one commits; the other retries. | **Time travel** — reading the table as of an earlier snapshot. Costs storage; priced against retention window. | **Manifest** — an index file listing data files plus column statistics, used by query planners to prune scans. |

# 2. Iceberg in one page

Iceberg is a **table format** — a specification for how Parquet files should be laid out in object storage so that a collection of them behaves like a database table. It is not a storage engine, a catalogue, or a query engine; each of those comes from somewhere else. This separation is the whole point of the format, and it is also why it generates operational work.

## 2.1 Physical layout

Every Iceberg table has four layers of files in S3. Figure 1 shows how they fit together. The important property is that *everything except the catalogue pointer* lives in S3 — including the history of every schema and every commit. The catalogue's only job is to say which *metadata.json* file is current right now. When the pointer is swapped atomically, readers see a new snapshot; when it fails, nothing changes. This is how Iceberg achieves ACID semantics on object storage.

### 2.2 Commits and snapshot isolation

Every write creates a new snapshot. Snapshots are immutable: a “delete” does not erase data, it writes a new snapshot that omits the deleted rows. Readers that opened against the old snapshot keep seeing the old data until they re-plan. This gives you snapshot isolation and time travel for free, but it also means **nothing is ever really deleted at write time**. Garbage collection is a separate, explicit maintenance job.

### 2.3 The catalogue’s role

Catalogues differ only in how they answer the “which *metadata.json* is current?” question. The choices are Iceberg REST (the open spec, spoken by AWS S3 Tables and Apache Polaris), AWS Glue (where the Glue table parameters point at the metadata file), Hive Metastore (legacy), and the proprietary catalogues inside Snowflake, Databricks Unity, and Dremio Arctic. Choosing a catalogue is the largest single decision in a migration because it determines *who runs the maintenance jobs*.

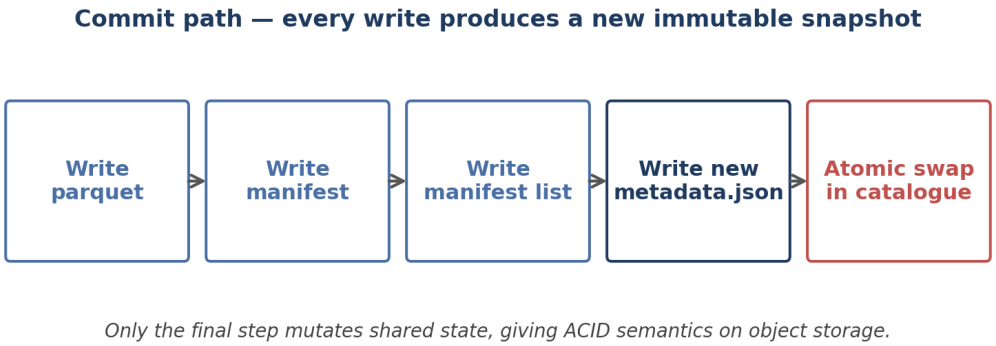



*Figure 2. The commit path. Four append-only writes to S3 followed by a single atomic swap in the catalogue. The swap is what makes commits transactional.*

## 3. The table lifecycle

Six stages span the life of every Iceberg table. Each carries a maintenance burden that *someone* — the engineering team, AWS, Snowflake, or Databricks — has to carry. Understanding which stages are cheap and which are expensive is the foundation of any honest cost comparison.

### 3.1 Write and append

A client writes Parquet file(s), writes a manifest referencing them, writes a manifest list referencing the new manifest plus the old ones, writes a new *metadata.json*, and asks the catalogue to swap the pointer. **The small-file problem is born here.** Every commit — no matter how small — produces at least one Parquet file. A firehose of one-row appends produces a firehose of one-row files, and read performance degrades superlinearly.

### 3.2 Read

Readers ask the catalogue for the current metadata, parse the manifest list, and use manifest-level column statistics to prune files before opening any Parquet. **Planning cost scales with the number of manifests, not the number of rows.** A neglected table can spend more time planning a query than scanning data — this is typically what a stakeholder is reporting when they say "*SELECT count(*)* takes 40 seconds on a tiny table." Figure 5 quantifies the effect.

### 3.3 Compaction

The cure for 3.1. A bin-pack rewrite merges many small Parquets into target-sized ones (typically 128–512 MB). Compaction comes in three flavours: simple bin-pack (always the baseline), sort-based (for tables queried with a consistent predicate), and cluster/Z-order (for analytical tables with several hot predicates). **Compaction is the single most load-bearing maintenance job.** Neglecting it produces slow reads, expensive scans, and unbounded manifest counts.

### 3.4 Snapshot expiration

*expire_snapshots* drops snapshot pointers older than a retention window; their data files are removed only if no other snapshot still references them. Without expiration, storage grows without bound: every compaction writes new files, while the pre-compaction files linger in old snapshots. A heavily maintained table can spend 10× its live data on retained snapshot history. Retention is a policy call — 7 to 30 days is typical — balancing time-travel coverage against storage cost.

### 3.5 Orphan file cleanup

*remove_orphan_files* deletes files in the table prefix that no manifest references. Orphans arise when writes fail partway through or when a non-Iceberg process writes into the prefix. **This is the maintenance job most likely to destroy data.** It must run with a grace period of at least 24–72 hours, so in-flight commits have a chance to land, and it must never be run on a prefix the team does not fully own.

### 3.6 Manifest rewrite and statistics

Over time, manifest churn — many small manifests each referencing a handful of files — slows planning even when the data files themselves are fine. *rewrite_manifests* merges them. Column statistics (per-column NDVs, histograms, theta sketches) are consumed by cost-based optimisers in Trino, Spark, and Snowflake. Iceberg supports the format; most engines do not yet write it. The absence shows up as poor join plans on large tables.

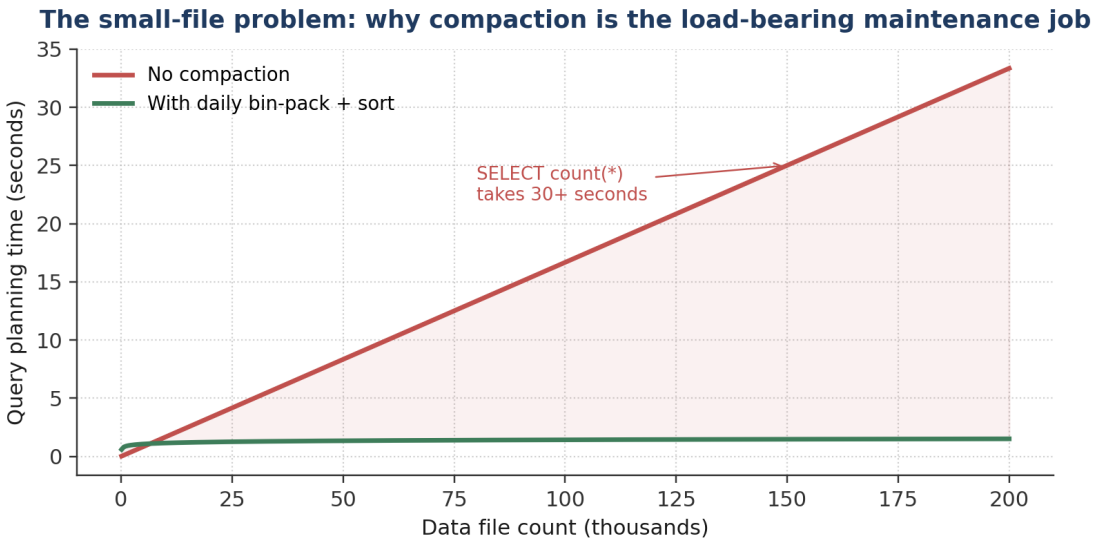



*Figure 5. Query planning cost as a function of data-file count. Without compaction, planning time scales linearly with file count; with disciplined compaction it grows logarithmically — the single clearest picture of why compaction is the load-bearing maintenance job.*

These six stages form the foundation of any honest cost comparison. The rest of this paper turns from *what* the maintenance jobs are to *who* pays for them — and at what level of service.

## 4. Why Snowflake became the default

Snowflake's pitch is not "a query engine." It is "we will operate a warehouse so you don't have to." The list of things that fall into that bucket is long, and none of it is visible on a day when it's all working. That silent service is precisely what teams forget they're consuming when they sketch a self-managed alternative on the back of a napkin.

| Service | What Snowflake does silently | What it costs if you do it yourself |
|---|---|---|
| Automatic compaction | Micro-partitions (~16 MB) continuously merged and re-clustered. | Dedicated Spark/EMR job or Lambda runner, scheduled per table, tuned per workload. |
| Automatic clustering | Declare a clustering key; Snowflake maintains it. | Sort-based compaction to maintain ordering; redone each compaction pass. |
| Automatic statistics | Used by the cost-based optimiser on every query. | You must write Iceberg statistics files; most engines don't. Bad join plans otherwise. |
| Snapshot management | Time-travel + fail-safe via a retention policy. | Scheduled *expire_snapshots*; audit trail to prove no accidental deletion. |
| Result & metadata cache | Identical queries return instantly; plan cache survives warehouse restarts. | Cache tier to build and operate (or accept the latency hit). |
| Materialised views | Maintained incrementally and automatically. | Pick a system (dbt incremental / Materialize) and wire it in. |
| Multi-writer concurrency | DML at scale without user intervention. | Single-writer invariant or real testing of optimistic-concurrency retries. |

| Service | What Snowflake does silently | What it costs if you do it yourself |
|---|---|---|
| Zero-copy clones | Metadata-only forks for dev/test. Cheap. | Branching catalogue (Nessie) or tag-based workflow; not free to operate. |
| Secure data sharing | Cross-account reads without copying data. | Lake Formation, IAM roles, or vendor equivalents; governance work. |

*Table 1. Snowflake's invisible service layer — and the replacement cost of each item if the team takes over.*

**What leaving Snowflake preserves:** bucket ownership, format portability, elastic compute decoupled from storage, per-query billing transparency, and the ability to query the same tables with any Iceberg-capable engine. **What leaving Snowflake removes:** every line in Table 1. Any cost case needs to say explicitly which of those we replace, which we accept degraded, and which we drop.

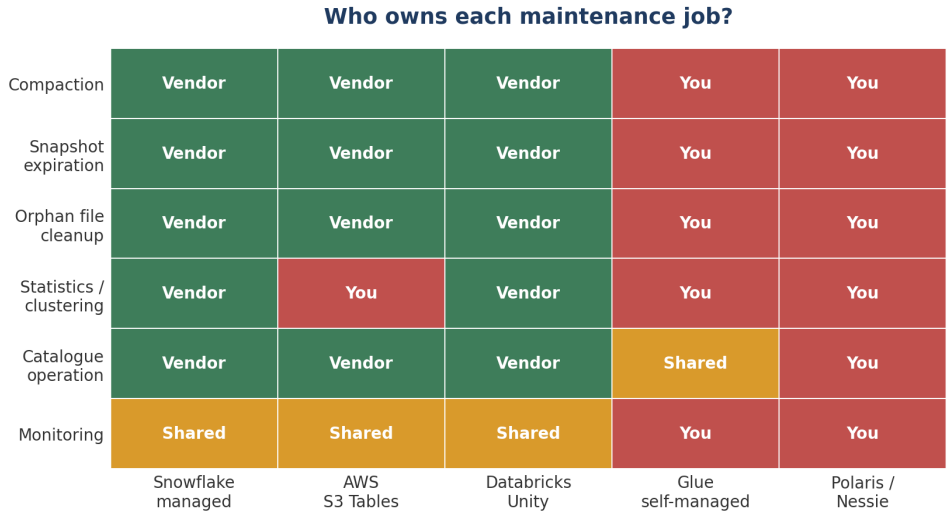



*Figure 3. Maintenance ownership across the five realistic catalogue options. Red cells are operational work the engineering team absorbs; amber cells are shared; green cells are vendor-managed.*

## 5. Options for Iceberg on S3

There are five realistic options on the table. All use the Iceberg-format data in S3; the differences are entirely in the catalogue and in who operates the maintenance jobs from §3. Because the file format is identical across rows, switching catalogues later is cheap — except for Snowflake-managed-internal, where the data lives in Snowflake's own storage.

| Option | Catalogue | Maintenance (compaction, expiration, orphans) | Statistics / clustering | Data ownership |
|---|---|---|---|---|
| **Snowflake-managed** | Snowflake internal (or external REST) | **Yes — all three, invisibly** | Auto clustering + search-opt indexes | Your bucket (external) or Snowflake's (internal) |
| **AWS S3 Tables** | Managed Iceberg REST | **Yes — background service** | Iceberg file stats only | Your account; bucket policy is service-managed |
| **Databricks Unity** | Unity (proprietary) | **Yes — Predictive Optimization** | Auto stats + liquid clustering | Your bucket; Unity governs access |

| Option | Catalogue | Maintenance (compaction, expiration, orphans) | Statistics / clustering | Data ownership |
|---|---|---|---|---|
| **Glue self-managed** | AWS Glue | **None — you run all three** | None automatic | Full — standard S3 bucket |
| **Polaris / Nessie** | Iceberg REST, self-hosted | **None — you run all three** | None automatic | Full — standard S3 bucket |

*Table 2. Iceberg catalogue options side-by-side. The three "Yes" rows are the managed choices; the two "None — you run it" rows trade operational burden for control.*

### Reading the table

Three rows are managed options (Snowflake, S3 Tables, Unity): the team pays for the service and maintenance is not on their plate. Two rows are bring-your-own-ops (Glue self-managed, Polaris/Nessie): cheaper on paper, maximum control, and the checklist in §8 is now theirs. Because the underlying Iceberg format is the same across all five, the team can switch catalogues later without rewriting data — the one exception being Snowflake-managed-internal, which keeps data in Snowflake's own storage.

> *"The Iceberg format is identical across every option. You can switch catalogues later. You cannot cheaply switch off Snowflake-managed-internal: the data lives in Snowflake's storage."*

## 6. An illustrative cost model

The five-year projection in Figure 4 compares three plausible paths for a workload with roughly 50 TB of active data and a steady query load. The inputs are calibrated to typical AWS and Snowflake list pricing, with self-managed totals that include a loaded engineering cost — approximately 1 to 1.5 full-time equivalents for maintenance, on-call, and incident response — because the maintenance jobs in §3 do not run themselves.

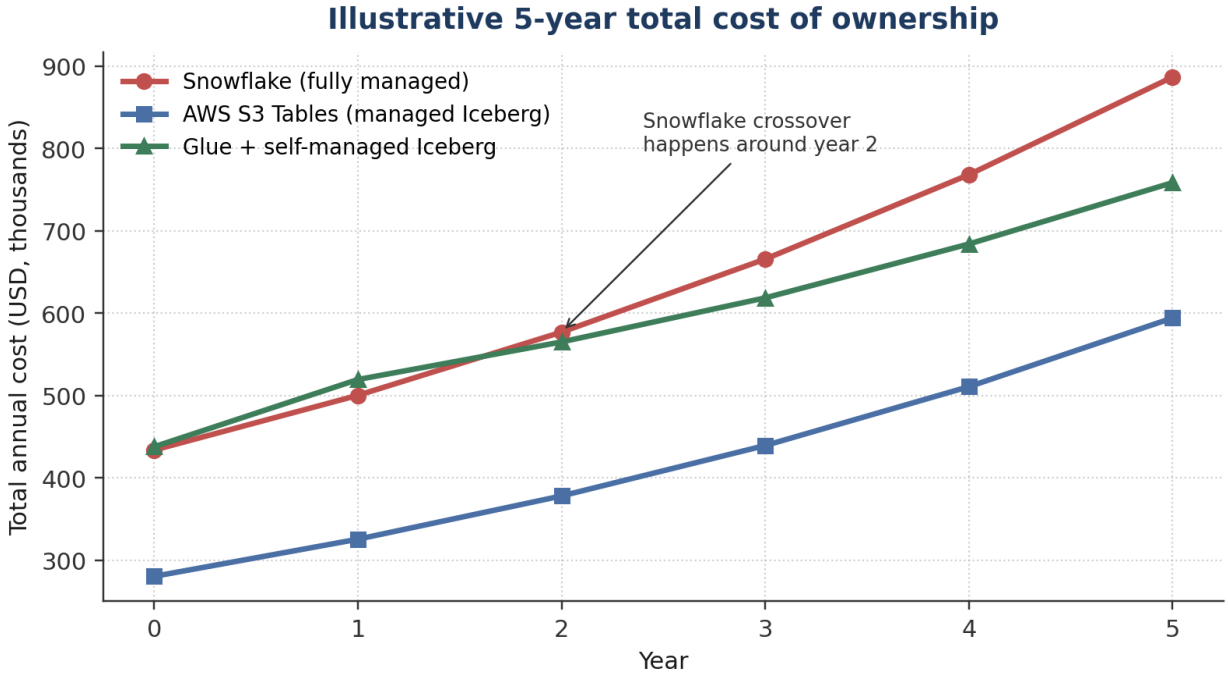



*Figure 4. Illustrative five-year total cost of ownership. Self-managed options include fully-loaded headcount and incident cost. Numbers are indicative and should be refined against real workload telemetry.*

### What the model assumes

| | |
|---|---|
| **Storage growth** | 25% per year across all options; same S3 cost base for self-managed paths. |
| **Compute scaling** | 15% per year, reflecting linear growth in query volume with data. |
| **Headcount** | $220k loaded per FTE in year 1, 6% annual growth. Glue-self-managed charges ~1.2 FTE; S3 Tables and Snowflake charge a fraction. |
| **Incident budget** | Glue path includes a front-loaded incident reserve reflecting the learning curve; declines as playbooks mature. |
| **Snowflake premium** | Reflects storage + credits pricing, with modest compound growth in credit usage. |

*Table 3. Assumptions behind the cost model. Any of these can be adjusted before the migration decision is made final.*

### What the model implies

Snowflake is the most expensive path after year two, but also the path that reliably delivers consistent performance with no maintenance surprises. S3 Tables sits comfortably in the middle: managed maintenance, open format, modest premium. Self-managed Glue is cheapest in steady state — but only once the team has absorbed the learning curve priced into year 1 and maintains operational discipline thereafter. **The cost advantage is real; it is not automatic.**

### What the model leaves out

Opportunity cost of engineering time. Vendor lock-in discount on Snowflake if negotiated. Non-linear Snowflake discounts at committed-use tiers. Unknown unknowns when engines behave differently against the same Iceberg tables. These should be revisited once concrete pricing quotes are in hand.

## 7. What the current repository does today

Both the managed and self-managed paths are wired up in parallel in the current codebase, so the team can compare them on real workloads without running a migration first. This is a deliberate choice: it gives us a week-by-week signal on the operational cost of each path before the strategic decision has to be made.

### 7.1 S3 Tables path (managed)

Terraform declares the table bucket, namespace, and one *aws_s3tables_table* per asset. The PyIceberg client loads the catalogue via the REST interface with SigV4 signing. AWS runs compaction, snapshot expiration, and orphan cleanup in the background. **There is no maintenance code in this path of the repository.**

### 7.2 Glue path (self-managed)

Terraform provisions a standard S3 bucket and Glue database, but tables are created lazily on first write because the Terraform provider collapses Iceberg types to Hive types and breaks required flags. The write Lambda performs a dual-write: every append is committed to both the S3 Tables path and the Glue path; if either fails, the source Parquet stays in the dropzone for retry. **No compaction, no snapshot expiration, no orphan cleanup is scheduled on the Glue side yet.** This is the gap that needs closing before any migration.

### 7.3 Known gaps

| | |
|---|---|
| **Compaction on Glue** | Small-file accumulation is already measurable on demo data. S3 Tables compacts automatically; Glue does not. |
| **Snapshot expiration** | S3 Tables handles its own; the Glue-side metadata directory grows unbounded. |
| **Orphan file cleanup** | Lower priority today because the write Lambda is the only writer and commits are atomic, but still a gap. |
| **Observability** | No commit-latency or manifest-count dashboards yet on either path. |

*Table 4. Gaps known to the team, not yet closed. None is a blocker for the pilot; all are blockers for a full cutover.*

## 8. Migration readiness checklist

Before any combination of the options in §5 replaces Snowflake in production, the team must own answers to all of the following. Any box still unchecked at cutover is an incident that has not yet filed itself.

| | | |
|---|---|---|
| ■ | **Compaction** | Scheduled job with a frequency (hourly or daily), concurrency cap, and cost budget. Self-managed Glue: pick a runner (Spark on EMR, Athena CTAS, or PyIceberg Lambda). S3 Tables: confirm the managed cadence meets latency requirements. |
| ■ | **Snapshot expiration policy** | Retention window expressed in both number of snapshots and wall-clock hours. Longer windows provide more time-travel coverage at the cost of storage. |
| ■ | **Orphan file cleanup** | Grace period of at least 24 hours. Must run with a lock preventing concurrent writes, or with the writer quiesced. Dry-run for the first several runs. |
| ■ | **Statistics and clustering strategy** | Identify the top three to five query patterns; pick clustering columns and a sort strategy per table. Decide whether statistics files are worth writing for the chosen engines. |
| ■ | **Monitoring** | At minimum: commit latency p95, manifest count per table, file count per partition, orphan-file-candidate count. Alert on manifests-per-table exceeding threshold — the “compaction is behind” signal. |
| ■ | **Catalogue backup / DR** | Glue: back up the catalogue database. Polaris or Nessie: back up the catalogue’s own storage. S3 Tables or Snowflake: vendor responsibility. |
| ■ | **Multi-writer invariant** | Either commit to a single-writer model and enforce it, or test optimistic-concurrency behaviour under the real write pattern. |
| ■ | **Access control model** | Write, read, and *expiration authority* are three different things. Whoever can run *expire_snapshots* can destroy time-travel history — scope it tightly. |

*Table 5. The pre-cutover checklist. Every row is also a line item in the TCO model — which is why self-managed Iceberg is “cheap” only when the whole list has an owner.*

## 9. Recommendation and next steps

The strategic case for Iceberg is clear: open format, engine portability, decoupled storage and compute, and (under discipline) a materially lower run rate than Snowflake. The operational case — who runs the maintenance — is the decision that actually matters. Based on the analysis above, we recommend the following sequence.

| | |
|---|---|
| **Quarter 1 — Measurement** | Run the dual-path dual-write for one quarter. Instrument both sides for commit latency, manifest count, and compaction cost. Capture a baseline against the current Snowflake workload. |
| **Quarter 2 — Close the gaps** | Build and schedule compaction, snapshot expiration, and orphan cleanup on the Glue side. Validate that query latency stays within the S3 Tables envelope. |
| **Quarter 3 — Pilot migration** | Migrate one non-critical workload to the chosen path. Run in parallel with Snowflake for four weeks. Compare cost, latency, and incident count. |
| **Quarter 4 — Decision** | Decide: retain Snowflake, cut over to S3 Tables, or cut over to self-managed Glue. The data from Q3 makes this an empirical call, not a directional one. |

*Table 6. Suggested four-quarter plan. Commits to measurement before migration, and keeps optionality open until the data arrives.*

## References

[1] Apache Software Foundation. *Apache Iceberg Specification v2.* Accessed April 2026. https://iceberg.apache.org/spec/

[2] Amazon Web Services. *Amazon S3 Tables documentation.* Accessed April 2026. https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables.html

[3] Snowflake, Inc. *Iceberg Tables in Snowflake.* Accessed April 2026. https://docs.snowflake.com/en/user-guide/tables-iceberg

[4] Databricks. *Unity Catalog and Delta Lake UniForm.* Accessed April 2026. https://docs.databricks.com/en/delta/uniform.html

[5] Internal: *architecture-review.md*. Strategic positioning of the open-format stack versus Snowflake, Databricks, and Athena.

[6] Internal: *terraform/main/terraform.tfvars*. Source of truth for tables provisioned in the current repository.